

Laboratorio #3 JavaScript y JSON

JavaScript (ECMAScript)

JavaScript es un lenguaje de programación interpretado, basado en objetos y de tipado débil, definido bajo el estándar ECMA-262. Mientras que el HTML define la estructura y el CSS la presentación, JavaScript se encarga del comportamiento y la interactividad en la web.

Ambiente de Ejecución de JavaScript:

JavaScript no es autosuficiente, depende de un entorno (como un navegador o un servidor) para realizar tareas de entrada/salida (I/O). Los navegadores pueden fungir como host para JavaScript, proveen objetos para manipular ventanas, formularios, eventos del DOM (clics, teclado) y cookies. Cada navegador tiene su propio motor de interpretación, lo que genera variaciones en el rendimiento.

Por otro lado, es un lenguaje versátil, lo que significa que puede funcionar como lenguaje procedural u orientado a objetos.

Características Técnicas Principales

- **Orientación a Objetos:** Basada en prototipos en lugar de clases tradicionales. Los objetos se crean dinámicamente añadiendo atributos a objetos vacíos.
- **Funciones de Primera Clase:** Las funciones son objetos, pueden tener métodos, ser pasadas como argumentos y crear clausuras léxicas (closures).
- **Tipado Dinámico:** El tipo de dato está asociado al valor y no a la variable, lo que ofrece gran flexibilidad.
- **Imperativo y Estructurado:** Sigue una lógica similar a C, pero con un ámbito de variables limitado a las funciones.
- **Funciones Variádicas:** Permiten recibir un número indefinido de parámetros a través del objeto `arguments`.
- **Expresiones Regulares:** Incluye soporte nativo para manipulación avanzada y sofisticada de texto.

Sintaxis básica de JavaScript

Declaración de variables:

```
let x; //Definición sin valor
const y = 2; //Definición y asignación de valor 2
```

Declaración de funciones (generales):

```
function sumar(a,b){
    return a+b;
}
```

Función recursiva :

```
function factorial(n){
```

```
    if ( n === 0 ){  
        return 1;  
    }  
    return n*factorial(n-1);  
}
```

Función anónima:

```
const sumar = function (a,b){  
    return a+b;  
}
```

Objeto (forma literal):

```
const persona = {  
    nombre: "Brendan Eich",  
    cedula: 111111,  
    edad: 52,  
    envejecer: function() { this.edad = this.edad + 1; }  
};
```

Objeto (forma dinámica):

```
let persona = {};  
persona.nombre = "Brendan Eich";  
persona.cedula = 111111;  
persona.edad = 52;  
persona.envejecer = function() { this.edad = this.edad + 1; };
```

Herencia (prototipo):

```
let persona = {  
    nombre: "Brendan Eich",  
    cedula: 111111,  
    edad: 52,  
    envejecer: function() { this.edad = this.edad + 1; }  
};  
var estudiante = Object.create(persona.prototype);  
estudiante.carnet = 123456;
```

JavaScript y HTML

Para agregar código JavaScript en un documento HTML se proveen tres diferentes formas:

Archivo Externo :

```
<script type = "text/javascript" src = "ruta/a1/archivo.js" />
```

Código explícito dentro del HTML:

```
<script type = "text/javascript" >
```



```
//código JavaScript  
</script>
```

Código en un Event Handler:

```
<element onEvent= "código JavaScript">
```

JSON

Es un formato ligero de intercambio de datos basado en texto plano. Aunque deriva de la notación de objetos de JavaScript (ECMA-262), es un formato independiente que destaca por ser fácil de leer y escribir tanto para humanos como para máquinas.

Los archivos de JSON usan la extensión .json y son del tipo MIME "application/json". Este formato es usado principalmente para serializar objetos JavaScript y transferir datos entre los servidores y las aplicaciones web.

Estructuras y Tipos de Datos

JSON se organiza fundamentalmente en dos estructuras: Colecciones de pares nombre/valor (objetos) y Listas ordenadas (arreglos). Sus valores permitidos son:

- **Números:** Enteros o decimales (ej. `123.45`).
- **Cadenas (Strings):** Secuencias de caracteres entre comillas dobles (ej. `"Hola"`).
- **Booleanos:** Valores lógicos `true` o `false`.
- **Null:** Representa un valor nulo.
- **Arreglos (Arrays):** Listas entre corchetes `[ ]` que pueden contener cualquier tipo de dato.
- **Objetos:** Conjuntos de pares `clave:valor` delimitados por llaves `{ }`.

Utilizar JSON en JavaScript

Método	Función	Resultado
<code>JSON.parse()</code>	Convierte una cadena de texto JSON en un objeto JavaScript.	Objeto usable en código.
<code>JSON.stringify()</code>	Convierte un objeto JavaScript en una cadena de texto JSON.	Texto listo para enviar al servidor.

Ejercicios

- a) Utilizar JavaScript para añadir funcionalidad a una página web. Para ello:
1. Cree un archivo HTML. Dentro de `<body>` agregue la siguiente estructura:

```
<div class="container">
  <h1>Laboratorio #3: JavaScript y JSON</h1>
  <h2>Funcionalidad con JavaScript</h2>
</div>
```

2. Agregue bajo la etiqueta <h2> el siguiente párrafo y un botón:

```
<p id="demo1">JavaScript permite modificar el contenido de un documento
HTML.</p>
<button class="btn" type="button">Aquí</button>
<br>
<br>
```

3. Añadiremos funcionalidad al botón, al presionarlo cambiaremos el contenido del párrafo. Para ello añadiremos un Event Handler al botón con el siguiente atributo:

```
onclick="document.getElementById('demo1').innerHTML = 'Hola JavaScript!'"
```

JavaScript tiende a tener una fuerte interacción con la API DOM, lo que permite acceder y manipular los documentos HTML y, por consiguiente, su comportamiento. En este caso estamos obteniendo un elemento del DOM por su id y modificando su HTML interno.

4. Abre el archivo HTML con *live server* y verifica que el texto cambia al presionar el botón.
5. Cambiemos ahora el estilo de los elementos de la página por medio de JavaScript. Para ello primero copie el siguiente HTML al final del contenedor

```
<div class="container">
<p id="demo2">Permíte cambiar el estilo de un documento HTML.</p>
<button class="btn" type="button" onclick="myFunction1()">Aquí</button>
<br>
<br>
```

Nuevamente estamos agregando un Event Handler directamente a la etiqueta HTML, sin embargo ahora estamos llamando a una función "myFunction1()" en lugar de escribir todo el código JS directamente dentro de la etiqueta.

6. Definamos ahora la función "myFunction1()", para ello añadimos la etiqueta <script> al final del <body> (debajo del contenedor <div class="container">).

```
<script>
function myFunction1() {
  var x = document.getElementById("demo2");
  x.style.fontSize = "25px";
  x.style.color = "violet";
}
```

```
}  
</script>
```

Presione el botón en la página y verifique que el estilo del texto cambia.

7. Añadamos ahora manejadores a eventos DOM utilizando solo JavaScript. Para ello añada el siguiente HTML al final del contenedor `<div class="container">`

```
<p id="demo3">Permite también agregar manejadores o handlers a eventos  
DOM.</p>  
<button class="btn" id="myBtn">Aquí</button>
```

8. Dentro de `<script>` seleccionamos el botón para la demo 3 por su id. Luego le añadimos un Event Listener para el evento de un clic y definimos una función.

```
document.getElementById("myBtn").addEventListener("click", function(){  
    alert("¡Hola Mundo!");  
});
```

9. En el navegador, compruebe que aparece la alerta al hacer clic sobre el botón.

b) Práctica con JSON. Para ello:

1. Descargue el archivo lab3.zip del campus virtual. Coloque el archivo content.json en el mismo directorio que su archivo html de la actividad anterior. Abre el archivo JSON y analiza su estructura.
2. Añade a tu archivo HTML la siguiente estructura al final del contenedor `<div class="container">`

```
<h2>Leer contenido de un archivo JSON</h2>  
<div id="demo5" style="display:none">  
    <h4 id="imgTitle"></h4>  
    <img id="myimg" src="" style="height: 400px">  
    <p id="imgDesc"></p>  
    <button class="btn" id="prev">Anterior</button>  
    <button class="btn" id="next">Siguiente</button>  
</div>  
<button class="btn" id="JSONBtn">Aquí</button>
```

3. Agregue funcionalidad al último botón. Para ello emplearemos el archivo JSON. No obstante, JavaScript no permite abrir archivos locales por temas de seguridad, pero para este laboratorio **evitaremos usar llamadas en segundo plano**. Por lo tanto, usaremos un truco para leer el contenido del archivo JSON de manera local desde JS. Agregue al inicio de la primera línea del archivo JSON lo siguiente:

```
const gallery =
```

Esto le permitirá tratar al archivo JSON como un objeto JS.

4. Agregue dentro de la etiqueta `<head>` del archivo HTML la siguiente etiqueta `<script>`:

```
<script type="text/javascript" src="content.json" defer></script>
```

5. Ahora se puede acceder al contenido del archivo JSON por medio de la variable "gallery" sin necesidad de definirla en nuestro script. Así, agregue al final de la etiqueta `<script>` dentro del `<body>` el siguiente código JS:

```
var galleryBtn = document.getElementById("JSONBtn");
var currentImg = 0;

function changeImg() {
    document.getElementById("imgTitle").innerHTML=
`${gallery[currentImg].title}`;
    document.getElementById("myimg").src= gallery[currentImg].url;
    document.getElementById("imgDesc").innerHTML=
`${gallery[currentImg].description}`;
}

galleryBtn.addEventListener("click", function() {
    document.getElementById("demo5").style.display = "block";
    changeImg();
    galleryBtn.style.display = "none";
});
```

6. Para agregar la funcionalidad de cambiar imágenes a los botones "Siguiente" y "Anterior", agregue las siguientes funciones a su código JS

```
document.getElementById("next").addEventListener("click", function() {
    currentImg = (currentImg+1)%gallery.length;
    changeImg();
});

document.getElementById("prev").addEventListener("click", function() {
    currentImg = currentImg-1;
    if (currentImg <0) currentImg = gallery.length-1;
    changeImg();
});
```



Reto 3 (at home)

Tomar las páginas web que se vienen trabajando en los retos y agregar dinamismo usando JavaScript y JSON, de manera de separar la estructura (HTML), de los datos y agregarle algo de comportamiento con un buscador. Para eso:

1. Eliminar de tu proyecto todos los directorios dummy. Hacer commit con el mensaje "sin dummies".
2. Bajarse el archivo reto3.zip del aula virtual y descomprimir este archivo en la raíz de tu proyecto. Luego de descomprimir vas a ver que hay otros directorios:
 - Varios directorios con nombres de CI que tienen fotos e información de otros perfiles.
 - Un directorio "conf" con 3 archivos JSON llamados configXX.json. Estos archivos tienen los textos del HTML que tienen que ver con la interfaz del usuario y podrían ser traducidos a otro idioma en caso de usar el mismo HTML para otro país. A estos datos los llamaremos datos de configuración. Los datos de configuración son textos que se muestran en la pantalla pero que no pertenecen al perfil del estudiante. Un ejemplo sería: "Mi color favorito es:" .
 - Un directorio "data" con un archivo JSON que tiene la lista de todos los perfiles.
3. Entrar al directorio con tu CI y sacar de la página *profile.html* los datos y crear un archivo JSON con tu información. Este archivo debe alojarse en el directorio con tu CI y debe llamarse "*profile.json*" y debe tener la misma estructura que en los otros directorios, pero con la información de tu perfil. En este archivo encontrarás campos para: nombre, descripción, color, libro, música, video juego, lenguajes, email. Además, campos nuevos: CI, género (femenino, masculino), fecha de nacimiento y extensión del archivo de la imagen.
4. Entrar al directorio "data" y agrega al principio del archivo "*index.json*" la información de acceso a tu perfil. Hacer commit con el mensaje "mis datos de perfil"
5. Mueve tu archivo "*profile.html*" del directorio con tu CI al directorio raíz, y elimina todos tus datos. Elimina todos los datos del archivo "*index.html*" también.
6. Editar el *index.html* y agregar un enlace que llame al JSON "*conf/configES.json*" en la etiqueta `<head>`.
7. Usar JavaScript para manipular los datos que están en los archivos JSON y agregar de forma automática la información de configuración en el HTML. El JavaScript debe quedar en el archivo externo "*js/index.js*", agregue en la etiqueta `<head>` el enlace a este archivo y asegúrese que el método de inicio se ejecute luego que cargue todo el HTML. Al tener todo listo, haga commit con el mensaje "configuración desde javascript"

Observe que los archivos JSON entregados empiezan con algo de código JavaScript: el nombre de una variable, asignación y llave ("const config = {" o "const profile = {" o "const profiles ="). Como **JavaScript NO permite abrir archivos directos desde la**

computadora personal por problemas de seguridad, y sólo se puede hacer importando desde un módulo (ES6+) o con llamadas en segundo plano desde un servidor web, **tenemos que hacer un pequeño truco para que este reto funcione en cualquier navegador**. El código JavaScript incluido en los JSON es una asignación del objeto JSON a una variable de JavaScript, **NO es parte de un archivo JSON como tal**, pero se necesita para que funcione este ejercicio.

8. Agregue código JavaScript para que se muestre la lista de perfiles según el JSON `"data/index.json"`, en vez de los dummies. La idea es que el `index.html` se vea como antes pero usando datos obtenidos de los archivos JSON. Agregue un event listener a las tarjetas de perfil de modo que al hacer clic sobre estas se redirija a `profile.html` y se agregue al URL un parámetro que guarde la cédula del perfil sobre el que se hizo clic. Al tener todo listo, haga commit con el mensaje "listado de perfiles dinámico usando javascript".
9. Edite el archivo `profile.html` y cree el archivo `js/profile.js` para que se muestre la información del perfil indicado desde el parámetro del URL que agregó en el paso anterior.

Asegúrese de que este archivo se inicialice igualmente con el archivo `"conf/configES.json"` para los textos estáticos de la página. Tome en cuenta que los archivos `configXX.json` tienen opciones plurales para algunos campos (por ejemplo: "Mis libros favoritos son:") esto es debido a que los perfiles pueden tener más de un valor para estos campos, debe garantizar que su código JavaScript maneje correctamente estos casos, mostrando todos los valores y seleccionando el texto singular o plural según aplique.

Al tener todo listo, haga commit con el mensaje "perfil con información dinámica".

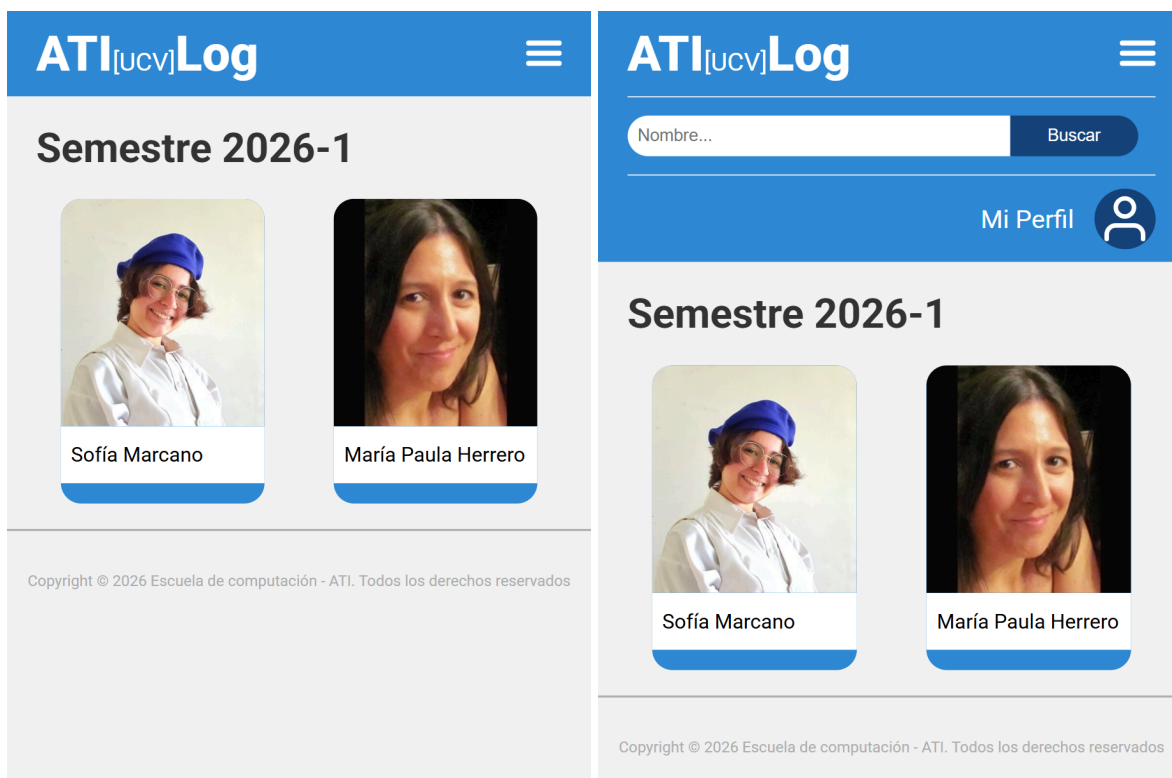
10. Permitir cambiar el lenguaje de la interfaz mediante un atributo del URL tanto en `index.html` como en `profile.html` usando JavaScript. Al tener todo listo, haga commit con el mensaje "cambio de lenguaje según atributo".
11. Agregue código JavaScript para hacer búsquedas sobre el listado de estudiantes. Lograr que cuando se escriba algo en el campo de texto del formulario de búsqueda vaya filtrando los alumnos del listado con aquellos estudiantes que tengan ese texto en su nombre. Si la búsqueda no consigue nada, debe aparecer un mensaje "No hay perfiles que tengan en su nombre: [query]", donde [query] es el texto que se busca. Tomar en cuenta que este texto, al ser de interfaz y sujeto a traducción, debe ser guardado en `"conf/configXX.json"`. Si no hay datos, la página se debe ver como se muestra en la imagen de abajo.



Además, hacer la búsqueda desde una página de perfil debe redirigir al listado y efectuar la búsqueda automáticamente.

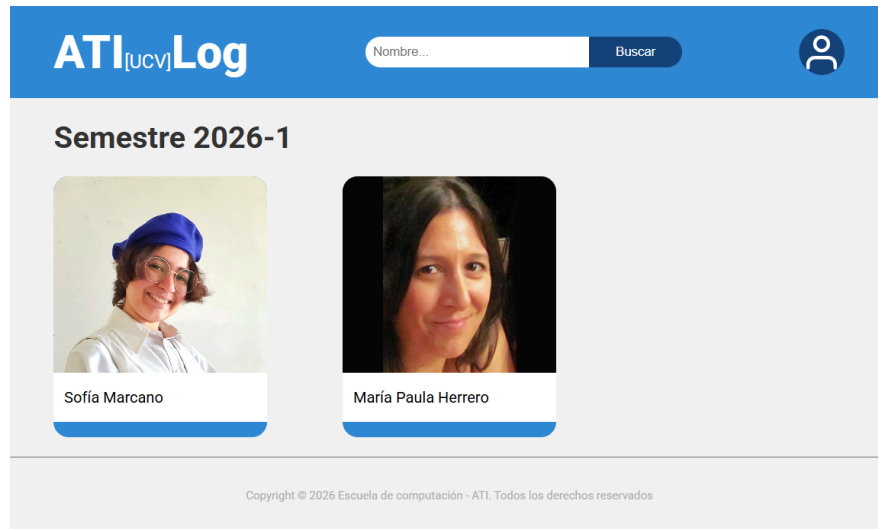
Al tener todo listo, haga commit con el mensaje “búsqueda de perfiles”.

12. Por último, agregue funcionalidad al botón del menú desplegable (para los dispositivos de pantallas pequeñas) . Al hacer clic sobre este botón se deben desplegar la barra de búsqueda y el botón de perfil, expandiendo el `<header>` verticalmente como se puede observar en las siguientes imágenes:



Tome en cuenta que:

- La barra de búsqueda debe mantener su funcionalidad.
- Volver a presionar el botón del menú debe ocultar nuevamente estos elementos.
- Esta funcionalidad aplica tanto para la página del listado como para las páginas de perfiles.
- Los diseños que no usen el botón del menú (los de tamaños de pantallas más grandes) no deben verse afectados por estos cambios de estilos dinámicos. Recordemos que de los retos anteriores el header debería verse como en la siguiente imagen:



Al finalizar, haga commit con el mensaje “menú desplegable”.

Al terminar, colocar en el reto del Aula Virtual el url del repositorio de GitHub que uso para hacer este ejercicio y una imagen del historial de Git Graph que muestra su Visual Code.